



Faculty of Engineering

Introduction to Machine Learning Lab Manual

Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: BRANCH:

SEMESTER: ACADEMIC YEAR:



Swaminarayan university
Publication

ISBN Number -



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester ME /
BE / DIPLOMA in has satisfactorily completed his
term working (Subject name) (subject code:)
during academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

Miss. Riya Parmar
SUBJECT IN-CHARGE

Asst. Prof. Shivani Gajjar
HOD

Dr. Bhavana Soni
DEAN

List of Practicals

Sr. No.	Title	Page No.	Date	Sign
1	Study and implement basic Python programming constructs used in machine learning applications.	3		
2	Perform data acquisition, visualization, and exploratory data analysis (EDA) using appropriate tools.	5		
3	Implement data preprocessing techniques including handling missing values, normalization, and feature scaling.	7		
4	Implement simple linear regression for prediction of continuous values.	9		
5	Implement multiple linear regression and evaluate model performance.	11		
6	Implement logistic regression for binary classification problems.	13		
7	Implement k-Nearest Neighbors (k-NN) algorithm for classification tasks.	15		
8	Implement decision tree algorithm for classification and prediction.	17		
9	Implement Naive Bayes classifier for probabilistic learning.	19		
10	Implement Support Vector Machine (SVM) for classification tasks.	21		
11	Perform clustering using k-Means algorithm for unsupervised learning.	23		
12	Evaluate machine learning models using performance metrics such as accuracy, precision, recall, and F1-score.	25		

PRACTICAL 1

Aim: Study and implement basic Python programming constructs used in machine learning applications.

Software / Tools Required:

Python 3.x, Jupyter Notebook / Google Colab, NumPy

Theory:

Python is the most widely used language for machine learning because of its simple syntax and rich ecosystem of libraries such as NumPy, Pandas, Matplotlib and Scikit-learn.

Before applying any ML algorithm, a programmer must be comfortable with core Python constructs: variables and data types, conditional statements, loops, functions, lists, tuples, dictionaries and array operations using NumPy.

NumPy provides the ndarray object which allows fast vectorized numerical computation, forming the base data structure for almost every ML library.

Program:

```
# Basic Python constructs used in Machine Learning

# 1. Variables and Data types
a = 10
b = 3.14
name = "Machine Learning"
is_valid = True
print(type(a), type(b), type(name), type(is_valid))

# 2. Conditional statement
x = 15
if x > 10:
    print("x is greater than 10")
else:
    print("x is 10 or less")

# 3. Loops
squares = []
for i in range(1, 6):
    squares.append(i ** 2)
print("Squares:", squares)

# 4. Function definition
def mean(values):
    return sum(values) / len(values)

print("Mean:", mean([2, 4, 6, 8, 10]))
```

```
# 5. List comprehension
even_numbers = [n for n in range(20) if n % 2 == 0]
print("Even numbers:", even_numbers)

# 6. Dictionary usage
student = {"name": "Aditi", "marks": 88}
print(f"{student['name']} scored {student['marks']} marks")

# 7. NumPy arrays (core data structure for ML)
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print("Array:", arr)
print("Mean using NumPy:", np.mean(arr))
print("Array + 10:", arr + 10)
print("Reshaped array:\n", arr.reshape(5, 1))
```

Output:

The program prints the data types of the declared variables, the list of squares [1, 4, 9, 16, 25], the mean of the sample list, the list of even numbers between 0 and 19, the student record, and the NumPy array along with its mean, element-wise addition and reshaped form.

Conclusion:

Thus, the basic Python programming constructs required for machine learning applications such as variables, conditionals, loops, functions and NumPy arrays were studied and implemented successfully.

PRACTICAL 2

Aim: Perform data acquisition, visualization, and exploratory data analysis (EDA) using appropriate tools.

Software / Tools Required:

Python 3.x, Pandas, Matplotlib, Seaborn, Scikit-learn

Theory:

Data acquisition is the process of collecting data from files, databases or APIs into a usable format, most commonly a Pandas DataFrame.

Exploratory Data Analysis (EDA) is used to understand the structure, distribution and relationships within a dataset before applying any ML algorithm. It involves checking data shape, summary statistics, missing values, and visualizing distributions using plots such as histograms, box plots, scatter plots and correlation heatmaps.

Libraries such as Pandas, Matplotlib and Seaborn are commonly used for data acquisition and visualization in Python.

Program:

```
# Data Acquisition, Visualization and Exploratory Data Analysis

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

# 1. Data Acquisition
iris = load_iris(as_frame=True)
df = iris.frame
df['species'] = df['target'].map(dict(enumerate(iris.target_names)))

print("First 5 rows:\n", df.head())
print("\nShape of dataset:", df.shape)
print("\nSummary statistics:\n", df.describe())
print("\nMissing values:\n", df.isnull().sum())

# 2. Visualization
plt.figure(figsize=(6, 4))
sns.histplot(df['sepal length (cm)'], kde=True)
plt.title("Distribution of Sepal Length")
plt.show()

plt.figure(figsize=(6, 4))
sns.boxplot(x='species', y='petal length (cm)', data=df)
plt.title("Petal Length by Species")
plt.show()

plt.figure(figsize=(6, 5))
```

```
sns.heatmap(df.drop(columns=['target', 'species']).corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

sns.pairplot(df, hue='species')
plt.show()
```

Output:

The program displays the first five rows, dataset shape (150, 6), summary statistics, and confirms there are no missing values. It generates a histogram of sepal length, a box plot of petal length grouped by species, a correlation heatmap, and a pair-plot showing relationships between all features colour-coded by species.

Conclusion:

Thus, data acquisition, visualization and exploratory data analysis were performed successfully on the Iris dataset using Pandas, Matplotlib and Seaborn.

PRACTICA 3

Aim: Implement data preprocessing techniques including handling missing values, normalization, and feature scaling.

Software / Tools Required:

Python 3.x, Pandas, NumPy, Scikit-learn

Theory:

Real-world data is often incomplete, noisy and inconsistent. Data preprocessing prepares raw data for machine learning algorithms.

Missing values can be handled by removing rows/columns (dropna) or by imputing them using the mean, median, or mode of the column.

Normalization (Min-Max scaling) rescales values to a fixed range, usually [0, 1], while Standardization (feature scaling using StandardScaler) transforms data to have zero mean and unit variance. These steps are essential because many ML algorithms are sensitive to the scale of input features.

Program:

```
# Data Preprocessing: Missing values, Normalization, Feature Scaling

import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Sample dataset with missing values
data = {
    'Age': [25, np.nan, 30, 22, np.nan, 40],
    'Salary': [50000, 54000, np.nan, 43000, 60000, 65000]
}
df = pd.DataFrame(data)
print("Original Data:\n", df)

# 1. Handling missing values using mean imputation
imputer = SimpleImputer(strategy='mean')
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
print("\nAfter Mean Imputation:\n", df_imputed)

# 2. Normalization (Min-Max Scaling) -> range [0, 1]
minmax_scaler = MinMaxScaler()
df_normalized = pd.DataFrame(minmax_scaler.fit_transform(df_imputed), columns=df.columns)
print("\nAfter Min-Max Normalization:\n", df_normalized)

# 3. Standardization (Feature Scaling) -> mean 0, std 1
standard_scaler = StandardScaler()
df_standardized = pd.DataFrame(standard_scaler.fit_transform(df_imputed), columns=df.columns)
print("\nAfter Standardization:\n", df_standardized)
```

Output:

The program displays the original dataset with missing values, the dataset after mean imputation, the normalized dataset with all values scaled between 0 and 1, and the standardized dataset with zero mean and unit variance.

Conclusion:

Thus, data preprocessing techniques including handling of missing values, normalization and feature scaling were implemented successfully.

PRACTICAL 4

Aim: Implement simple linear regression for prediction of continuous values.

Software / Tools Required:

Python 3.x, Pandas, NumPy, Matplotlib, Scikit-learn

Theory:

Simple Linear Regression models the relationship between a single independent variable (X) and a dependent variable (Y) by fitting a straight line: $Y = b_0 + b_1 \cdot X$.

The coefficients b_0 (intercept) and b_1 (slope) are estimated by minimizing the sum of squared errors between the predicted and actual values (Ordinary Least Squares method).

It is used for predicting continuous outcomes, such as predicting salary based on years of experience.

Program:

```
# Simple Linear Regression

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

# Sample data: Years of Experience vs Salary
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([35000, 40000, 46000, 50000, 55000,
              62000, 68000, 74000, 80000, 88000])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercept (b0):", model.intercept_)
print("Slope (b1):", model.coef_[0])
print("R2 Score:", r2_score(y_test, y_pred))
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))

# Predict salary for 11 years of experience
new_exp = np.array([[11]])
print("Predicted salary for 11 years experience:", model.predict(new_exp)[0])

plt.scatter(X, y, color='blue', label='Actual data')
plt.plot(X, model.predict(X), color='red', label='Regression line')
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.title("Simple Linear Regression")
plt.legend()
plt.show()
```

Output:

The program prints the intercept, slope, R2 score and mean squared error of the fitted model, predicts the salary for 11 years of experience, and plots the regression line along with the actual data points.

Conclusion:

Thus, simple linear regression was implemented successfully to predict a continuous value (salary) based on a single feature (years of experience).

PRACTICAL 5

Aim: Implement multiple linear regression and evaluate model performance.

Software / Tools Required:

Python 3.x, Pandas, NumPy, Scikit-learn

Theory:

Multiple Linear Regression extends simple linear regression to model the relationship between one dependent variable and two or more independent variables: $Y = b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n$.

Model performance is evaluated using metrics such as R2 score (coefficient of determination), Mean Absolute Error (MAE), Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).

It is useful when the target variable depends on multiple factors simultaneously, such as house price depending on area, number of rooms and location.

Program:

```
# Multiple Linear Regression

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Sample dataset: House price prediction
data = {
    'Area_sqft': [1200, 1500, 1800, 2000, 2300, 2600, 3000, 3400, 3800, 4200],
    'Bedrooms': [2, 3, 3, 3, 4, 4, 4, 5, 5, 5],
    'Age_years': [10, 8, 5, 12, 3, 7, 2, 6, 1, 4],
    'Price_lakh': [45, 55, 62, 65, 78, 82, 95, 105, 118, 128]
}
df = pd.DataFrame(data)

X = df[['Area_sqft', 'Bedrooms', 'Age_years']]
y = df['Price_lakh']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Coefficients:", dict(zip(X.columns, model.coef_)))
print("Intercept:", model.intercept_)
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score:", r2_score(y_test, y_pred))
```

Output:

The program displays the regression coefficients for each independent feature, the intercept, and the evaluation metrics MAE, MSE, RMSE and R2 score for the fitted multiple linear regression model.

Conclusion:

Thus, multiple linear regression was implemented successfully and the model performance was evaluated using standard regression metrics.

PRACTICAL 6

Aim: Implement logistic regression for binary classification problems.

Software / Tools Required:

Python 3.x, Pandas, NumPy, Scikit-learn

Theory:

Logistic Regression is a classification algorithm used to predict the probability of a binary outcome (0 or 1) using the sigmoid function: $p = 1 / (1 + e^{-(b_0 + b_1X)})$.

Unlike linear regression, it outputs probabilities that are mapped to class labels using a threshold (commonly 0.5).

Model performance is evaluated using a confusion matrix, accuracy, precision, recall and F1-score.

Program:

```
# Logistic Regression for Binary Classification

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_breast_cancer()
X, y = data.data, data.target    # 0 = malignant, 1 = benign

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred, target_names=data.target_names))
```

Output:

The program prints the accuracy of the logistic regression classifier, the confusion matrix showing true/false positives and negatives, and a full classification report with precision, recall and F1-score for both classes.

Conclusion:

Thus, logistic regression was implemented successfully for a binary classification problem (breast cancer diagnosis) and evaluated using standard classification metrics.

PRACTICAL 7

Aim: Implement k-Nearest Neighbors (k-NN) algorithm for classification tasks.

Software / Tools Required:

Python 3.x, Pandas, NumPy, Scikit-learn

Theory:

k-Nearest Neighbors is a simple, non-parametric, instance-based learning algorithm. It classifies a new data point based on the majority class among its 'k' nearest neighbours in the feature space, typically using Euclidean distance.

The choice of k is important: a small k makes the model sensitive to noise, while a large k can smooth out decision boundaries excessively.

k-NN requires feature scaling since it depends heavily on distance calculations.

Program:

```
# k-Nearest Neighbors (k-NN) Classification

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred, target_names=iris.target_names))

# Finding the best value of k
for k in range(1, 11):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))
    print(f"k = {k}, Accuracy = {acc:.3f}")
```

Output:

The program prints the accuracy, confusion matrix and classification report of the k-NN classifier with k=5 on the Iris dataset, followed by a comparison of accuracy for k values from 1 to 10.

Conclusion:

Thus, the k-Nearest Neighbors algorithm was implemented successfully for the classification of the Iris dataset.

PRACTICAL 8

Aim: Implement decision tree algorithm for classification and prediction.

Software / Tools Required:

Python 3.x, Pandas, Scikit-learn, Matplotlib

Theory:

A Decision Tree is a tree-structured classifier where internal nodes represent tests on features, branches represent outcomes of the test, and leaf nodes represent class labels.

It splits the dataset based on criteria such as Gini Impurity or Information Gain (Entropy) to create the purest possible child nodes at each step.

Decision trees are easy to interpret and visualize but can overfit if not pruned (controlled using max_depth or min_samples_leaf).

Program:

```
# Decision Tree Classifier

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred, target_names=iris.target_names))

plt.figure(figsize=(12, 8))
plot_tree(model, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.title("Decision Tree Visualization")
plt.show()
```

Output:

The program prints the accuracy, confusion matrix and classification report of the decision tree classifier, and displays a visual representation of the trained decision tree showing the splitting conditions at each node.

Conclusion:

Thus, the decision tree algorithm was implemented successfully for classification and its structure was visualized.

PRACTICAL 9

Aim: Implement Naive Bayes classifier for probabilistic learning.

Software / Tools Required:

Python 3.x, Pandas, Scikit-learn

Theory:

Naive Bayes is a probabilistic classifier based on Bayes' Theorem, which assumes that all features are conditionally independent given the class label (the 'naive' assumption).

It calculates the posterior probability $P(\text{class} \mid \text{features})$ for each class and assigns the data point to the class with the highest probability.

Gaussian Naive Bayes is used when features are continuous and assumed to follow a normal distribution, and is popular for its speed and good performance on high-dimensional data such as text classification.

Program:

```
# Naive Bayes Classifier

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

wine = load_wine()
X, y = wine.data, wine.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred, target_names=wine.target_names))
```

Output:

The program prints the accuracy, confusion matrix and classification report of the Gaussian Naive Bayes classifier trained on the Wine dataset.

Conclusion:

Thus, the Naive Bayes classifier was implemented successfully for a probabilistic classification task.

PRACTICAL 10

Aim: Implement Support Vector Machine (SVM) for classification tasks.

Software / Tools Required:

Python 3.x, Pandas, Scikit-learn

Theory:

Support Vector Machine is a supervised learning algorithm that finds the optimal hyperplane which best separates data points of different classes with the maximum margin.

Data points closest to the hyperplane are called support vectors, as they determine the position and orientation of the hyperplane.

For non-linearly separable data, SVM uses kernel functions (linear, polynomial, RBF) to map data into a higher-dimensional space where a linear separator can be found.

Program:

```
# Support Vector Machine (SVM) Classification

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = SVC(kernel='rbf', C=1.0, gamma='scale')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred, target_names=data.target_names))

# Comparing different kernels
for k in ['linear', 'poly', 'rbf', 'sigmoid']:
    m = SVC(kernel=k)
    m.fit(X_train, y_train)
    acc = accuracy_score(y_test, m.predict(X_test))
    print(f"Kernel = {k}, Accuracy = {acc:.3f}")
```

Output:

The program prints the accuracy, confusion matrix and classification report of the SVM classifier with an RBF kernel, followed by an accuracy comparison across linear, polynomial, RBF and sigmoid kernels.

Conclusion:

Thus, the Support Vector Machine algorithm was implemented successfully for classification and the effect of different kernels was compared.

PRACTICAL 11

Aim: Perform clustering using k-Means algorithm for unsupervised learning.

Software / Tools Required:

Python 3.x, Pandas, NumPy, Matplotlib, Scikit-learn

Theory:

k-Means is an unsupervised learning algorithm used to partition a dataset into 'k' clusters, where each data point belongs to the cluster with the nearest mean (centroid).

The algorithm iteratively assigns points to the nearest centroid and updates centroids until convergence, minimizing the within-cluster sum of squares (WCSS).

The optimal number of clusters is often determined using the Elbow Method, which plots WCSS against different values of k and looks for the 'elbow' point.

Program:

```
# k-Means Clustering

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Generate sample unlabeled data
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=42)

# Elbow method to find optimal k
wcss = []
for k in range(1, 10):
    km = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
    km.fit(X)
    wcss.append(km.inertia_)

plt.plot(range(1, 10), wcss, marker='o')
plt.xlabel("Number of clusters (k)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()

# Applying k-Means with chosen k = 4
kmeans = KMeans(n_clusters=4, init='k-means++', random_state=42, n_init=10)
labels = kmeans.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            s=200, c='red', marker='X', label='Centroids')
plt.title("k-Means Clustering (k=4)")
plt.legend()
plt.show()
```

Output:

The program plots the elbow curve to help determine the optimal number of clusters, and then displays a scatter plot of the data points coloured by their assigned cluster along with the final cluster centroids marked in red.

Conclusion:

Thus, k-Means clustering was implemented successfully to group unlabeled data into meaningful clusters using unsupervised learning.

PRACTICAL 12

Aim: Evaluate machine learning models using performance metrics such as accuracy, precision, recall, and F1-score.

Software / Tools Required:

Python 3.x, Pandas, Scikit-learn, Matplotlib, Seaborn

Theory:

Evaluating a classification model requires more than just accuracy, especially when the dataset is imbalanced.

Accuracy is the ratio of correctly predicted observations to total observations. Precision is the ratio of true positives to all predicted positives (measures exactness). Recall (Sensitivity) is the ratio of true positives to all actual positives (measures completeness). F1-score is the harmonic mean of precision and recall, providing a balance between the two.

These metrics are derived from the confusion matrix, which tabulates True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN).

Program:

```
# Model Evaluation using Performance Metrics

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)

import seaborn as sns
import matplotlib.pyplot as plt

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall    :", recall_score(y_test, y_pred))
print("F1-score :", f1_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred,
    target_names=data.target_names))

cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=data.target_names, yticklabels=data.target_names)
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")  
plt.title("Confusion Matrix")  
plt.show()
```

Output:

The program prints accuracy, precision, recall and F1-score for the trained classifier along with a full classification report, and displays a heatmap of the confusion matrix comparing predicted and actual class labels.

Conclusion:

Thus, a machine learning model was evaluated successfully using accuracy, precision, recall and F1-score, providing a complete picture of its classification performance.